

WEEK 10 · STATE, RENDER, ONE SOURCE OF TRUTH

ONE PLACE WHERE THINGS ARE TRUE

Last week the page started answering. Today it stops holding opinions of its own.

WHERE WE ARE

Week 10 of 12 (week 13 is the exam). An application that works, and a bug in it that cannot be fixed by fixing it.

THE NEXT 180 MINUTES

Four cycles, you typing nearly every piece (the long blocks are marked **copy this**): three layers · a handler that describes · one derivation · a filter from Claude. Break at 85, the surgery at 120, the assignment at 160.

WHAT THIS UNLOCKS

Every project in this room ends up the same shape — which is the only reason the exam in week 13 can be the same for all of you.

This is the week the course names its architecture.

TARGET · MINUTE 0

THIS IS WHERE YOU WILL BE AT MINUTE 180

the pantry, taken apart into three layers – it looks exactly like the one you start from

reloadopen

המזווההרשימהפריטסיכום

המזווה

שם

כמות

מדף

הוסף

חיפוש

מדף

מיון

לפי שם

הכול

מוצג: 6

המזווההרשימהפריטסיכום

המזווה

שם

כמות

מדף

הוסף

חיפוש

מדף

מיון

לפי שם

הכול

מוצג: 6

targets/week-10.png · 1280

המזווההרשימהפריטסיכום

המזווה

שם

375 · the same shape, narrower

► **THE POINT** — the screen does not change today. What changes is that the four faults in the starter **STOP BEING POSSIBLE**: one object, one painter, handlers that only describe. You build each piece once in a cycle first, on a different app: the class lending library.

AI THIS WEEK

PARTNER. IT WRITES; YOU REVIEW IT AGAINST THREE LAWS.

LAYER • WEEKS 8–12 • PARTNER

ALLOWED

Claude writes code — in Claude Code on the folder, or in the chat with the files pasted — **against a rule you wrote into the prompt**, and you read what came back before you run it (cycle 4; cycle 2's undo).

NOT ALLOWED

Code you cannot explain line by line. A refactor that also “fixes” things you did not ask for. **Any AI in cycles 1 and 3's “you do” and in the surgery** — alone, offline, like week 13.

NO SUBSCRIPTION?

Everything works in the free chat, and nothing in the course depends on a paid plan. Paste the files, copy back what returns, run the same review. Same grade.

A BUG YOU CANNOT FIX BY FIXING IT

WHAT HAPPENS

Remove a book. Add a different one. **The removed book is back.**

The rows on the screen and the array `books` in memory are two copies of one fact.

THE OBVIOUS FIX

The remove handler deletes the row from the screen and forgets the array. So make it delete from the array too.

WHY THAT IS NOT A FIX

Now **two** things have to be updated on every removal, by whoever writes the next handler. You have made the bug harder to hit, not impossible.

Every bug this week is the same bug: something is true in two places.

TWO COPIES, AND THE DISTANCE BETWEEN THEM

left: the array · right: the rows · far right: how far apart, computed

↶ reloadopen ↷

ONE LIST, TWO PLACES IT IS WRITTEN DOWN

push to the array only

append a row only

remove a row only

compare the two

redraw from the array

reset

THE ARRAY — ITEMS

[
 "flour",
 "olive oil",
 "almond milk"
]

THE SCREEN — #LIST

flour

olive oil

almond milk

HOW FAR APART THEY ARE

0

the two agree

Nothing on this page repairs the drift, and that is the point: every repair you can think of is one more line that one more handler has to remember, forever.

► **PRESS “APPEND A ROW ONLY” THREE TIMES, THEN DECIDE WHAT THE ARRAY HOLDS.** Decide before you press compare.

THE ONE-WAY LOOP

```
1 state → render(state) → the screen
2   ▲
3   | handlers ←
```

STATE

One JavaScript object that holds everything the page needs to know.

RENDER(STATE)

One function that draws the whole screen from that object, and from nothing else.

A HANDLER

Code that runs when the user acts. It changes `state`; it draws nothing.

Data flows one way round. The screen never writes back: the one-way loop.

RENDER(STATE) IS A FUNCTION OF ITS ARGUMENT

edit the object on the left, press render, and there is no third input

[↺ reload](#)[open ↗](#)

THE SCREEN IS A FUNCTION OF THIS OBJECT, AND OF NOTHING ELSE

[render\(state\)](#)[render twice, compare the two](#)[delete a row straight from the DOM](#)[subscribe without the first draw](#)[reset](#)

STATE — EDIT IT

the whole application, as one object

```
{
  "items": [
    {
      "id": "a1",
      "name": "flour",
      "qty": 2
    },
    {
      "id": "a2",
      "name": "olive oil",
```

valid JSON

THE SCREEN — RENDER(STATE)

showing 3 of 3 · renders so far: **1**

almond milk

4

flour

2

olive oil

1

nothing measured yet

There is no button here that changes the screen without changing the object. That is not a limitation of the demo — it is the whole rule, and everything else this week follows from it.

► “DELETE A ROW STRAIGHT FROM THE DOM” — IS THE ITEM GONE? Decide first. Then press render.

THREE LAWS, AND EVERYTHING TODAY FOLLOWS FROM THEM

1 · ONE TRUTH

If it is on screen, it came from `state`. If it is not in `state`, it is not real.

2 · ONE PAINTER

Only `render(state)` writes to the screen. Nothing else, ever.

3 · ONE DIRECTION

Handlers change `state`. Nothing reads the screen to find out what is true.

► A fourth follows from all three and it is the one people lose marks on:

ANYTHING THAT CAN BE COMPUTED FROM STATE IS COMPUTED, NEVER STORED (cycle 3 calls it *derived*).

WHAT EACH LAYER IS FORBIDDEN TO DO

FILE	ITS JOB	WHAT IT MAY NEVER DO
js/state.js	the object, and the functions that answer questions about it	touch the DOM
js/render.js	state in, screen out: one entry point, <code>render</code> , that calls one small function per part of the page; each returns at once when its part is not on this page	change state · add listeners · read the screen
js/events.js	turn a user action into a change of state; two exceptions, both for form messages: <code>readForm</code> in the cycle writes the <code>#form-msg</code> text, and the form's <code>fail()</code> helper (in your starter) shows an error beside the field	draw anything
js/app.js	wire the three together, once	grow

► The third column is the useful one: **LAYERS ARE DEFINED BY THEIR PROHIBITIONS**. Each ban has a price. A render that changes state calls itself. A listener (a function the browser calls when the user acts) added inside render is added again on every redraw. A render that reads the screen has made the screen a second truth.

WHY THE SPLIT MAKES EVERY FUTURE CHANGE LOCAL

the same 47 lines · as one file and as three · pick a change requestreloadopen

THE SAME 46 LINES, AS ONE FILE AND AS THREE, AND WHERE A CHANGE LANDS

change request

show the shelf name on every row

clear the highlight

ONE FILE — APP.JS

9 to change · 47 to read

one file, so there is no line you can be sure the change does not touch.
app.js

```
1 import { SEED } from './items.js';
2
3 let items = SEED.map((item) => ({ ...item }));
4 let query = '';
5 let sort = 'name';
6
7 const list = document.querySelector('#list');
8 const count = document.querySelector('#count');
9 const queryField = document.querySelector('#query');
10 const sortField = document.querySelector('#sort');
11 const form = document.querySelector('#item-form');
12
13 function visible() {
14   return items
15     .filter((item) => item.title.includes(query))
```

THREE FILES

10 to change · 19 to read

a change to how a row LOOKS. It has one home. One file opened.
state.js

```
1 let state = { items: [], query: '', sort: 'name' };
2 const listeners = new Set();
3
4 export const getState = () => state;
5
6 export function setState(patch) {
7   state = { ...state, ...patch };
8   for (const listener of listeners) listener(state);
9 }
10
11 export function selectVisible(state) {
12   return state.items
13     .filter((item) => item.title.includes(state.query))
14     .sort((a, b) =>
15       state.sort === 'number'
```

▶ Do not watch the “lines to change” number — it is nearly the same. Watch **LINES TO READ**.

WHY A LISTENER INSIDE RENDER MULTIPLIES — AND WHEN IT WOULD NOT

AN ARROW, TEN TIMES

```
1 for (let i = 0; i < 10; i++) {  
2   button.addEventListener('click', () => { n += 1; });  
3 } // one click: n === 10
```

THE SAME FUNCTION, TEN TIMES

```
1 function onClick() { n += 1; }  
2 for (let i = 0; i < 10; i++) {  
3   button.addEventListener('click', onClick);  
4 } // one click: n === 1
```

► The DOM refuses to add **THE SAME FUNCTION** twice for the same type — and an arrow written inside `render` is a **NEW** function on every draw. The fix is not a named function: it is the listener in `events.js`, registered once.

FOUR FILES: EACH ONE HANDS ITS NAMES TO THE OTHERS

STATE.JS · A DOOR OUT

```
1 export const getState = () => state;  
2 export function setState(patch) { ... }
```

`export` puts a name on the file's door. Whatever is not exported stays private to the file.

RENDER.JS · A DOOR IN

```
1 import { getState } from './state.js';  
2 // an address, not a name: keep the ./ and the .js
```

`import` takes a name through that door, by the address of the file.

```
1 <script type="module" src="js/app.js"></script>
```

► The page loads only `app.js`, as a module; it imports the rest. A browser refuses to load modules from a double-clicked file (`file://`): open the folder with **LIVE SERVER**, the rule from week 1.

THE START OF STATE.JS: A FUNCTION, ONE VARIABLE, ONE SET

```
1 function initialState() {  
2   return { books: [] };  
3 }  
4  
5 let state = initialState();  
6 const listeners = new Set();
```

INITIALSTATE()

A function that builds a fresh object every time it is called — not one object written out once and shared.

LET STATE

The only variable in the whole application that holds the truth.

NEW SET()

A collection that keeps each value once: `add` puts one in, `delete` takes one out, `for...of` visits each. Here it holds every **listener**: a function waiting to be told about each change.

A FUNCTION, NOT AN OBJECT LITERAL

SHARED BY ACCIDENT

```
1 export const INITIAL = {  
2   books: [],  
3   query: '',  
4 };
```

One object. Every page that imports it gets **the same one**, and one mutation leaks into the next.

FRESH EVERY TIME

```
1 function initialState() {  
2   return {  
3     books: [],  
4     query: '',  
5   };  
6 }
```

Called once per load. **Week 8's value-versus-reference**, arriving where it costs something.

ONE DOOR IN, AND IT IS THE REASON EVERYTHING ELSE WORKS

```
1 export const getState = () => state;
2 export function setState(patch) {
3   state = { ...state, ...patch };
4   for (const listener of listeners) listener(state);
5 }
```

► The new state first, then every listener is handed it. Because **NOTHING OUTSIDE THIS FILE ASSIGNS TO STATE**, “did anybody change the books without redrawing?” has a permanent answer of no.

`subscribe(listener)` adds a function to that Set and returns a function that removes it. A function remembers the variables around where it was written (a **closure**), which is how the returned function still knows which listener to remove.

THE WHOLE APPLICATION, IN FOUR LINES

```
1 seed(BOOKS);           // copy the six books from data.js into state
2 subscribe(render);      // state -> screen, on every change, forever
3 wire();                 // screen -> state: attach the handlers, once
4 render(getState());     // draw once, because subscribing does not fire
```

▶ `seed` is a function in `state.js` and `wire` one in `events.js`; you write both today. `BOOKS` is the starting array that `data.js` exports.

Line 4 is the one people forget, and the page it produces looks exactly like a broken render function.

SEED() COPIES — ONE LEVEL DEEP

WHAT CYCLE 1 TYPES

```
1 books.map((book) => ({ ...book })))
```

A new array of new objects. The seed file and the state no longer share an object — enough, because every field of a book is a string or a number.

WHEN A FIELD IS ITSELF AN OBJECT

```
1 const seed = [{ id: 'b1', tags: ['x'] }];  
2 const copy = seed.map((b) => ({ ...b }));  
3 copy[0].tags.push('y');  
4 seed[0].tags; // ['x', 'y'] — shared
```

Spread copies one level. `structuredClone(seed)` copies all the way down — measured: a push into the clone leaves the seed untouched.

CYCLE 1 · 27 MIN

FROM ONE FILE TO THREE LAYERS, AND THE LINE EVERYBODY FORGETS

The class lending library, one file, working — and its first fault: remove a book, add one, and the removed book is back. You move it into `state.js` · `render.js` · `events.js`, wire it in four lines, and the screen ends exactly as it started. Fault included.

1 I DO

14 MIN

7 on the slides before this one, 7 in the editor · live-code/01-monolith-to-three-layers · state.js with one door · render.js and events.js (the moved blocks: copy this) · app.js in three lines (wrong) · the fourth line

2 WE DO

4 MIN

you type the same steps, moved blocks pasted from the old app.js · compare with checkpoint 1 · "document" in state.js: zero

3 YOU DO

7 MIN

the second page: book.html on the same app.js, the id from the address, null is an empty state

4 DEBRIEF

2 MIN

a blank page and a clean console is not a broken render · ↗ depth

YOU DO · AI-FREE

CHECKPOINT 1

YOUR SCREEN SHOULD LOOK LIKE THIS

ספריית ההשאלה

הרשימה

ספר

ספריית ההשאלה של הכיתה

שם הספר

עותקים

מדור

הוסף

מוציגים: 6

המפה והים	מסע	2	1+	הסר
אור בחלון	רומן	1	1+	הסר
שעון החול	מדע	3	1+	הסר
גשר על הירקון	רומן	1	1+	הסר
כוכבים בצהריים	מדע	4	1+	הסר
שביל החלב	מסע	2	1+	הסר

Compare — the page did not move a pixel, and that is the point. What changed is next to it: `js/state.js` has no `document`, `js/render.js` no `addEventListener`, `js/app.js` is four lines. Remove a book and add one: it still comes back. That is cycle 2.

YOU DO · AI-FREE · BOOK.HTML LOADS THE SAME JS/APP.JS

ONE RENDER, TWO PAGES, AND AN ID THAT IS NOT THERE

STATE.JS · ONE BOOK, OR NULL

```
1 export function selectOne(state, id) {  
2   return state.books.find((book) => book.id === id) ?? null;  
3 }
```

RENDER.JS · A NEW REGION

```
1 function renderBook(state) {  
2   const mount = document.querySelector('#book');  
3   if (!mount) return;           // not this page: do nothing  
4   const id = new URLSearchParams(location.search).get('id');  
5   const book = selectOne(state, id); // the book, or null  
6   if (book === null) {  
7     // an empty state: a sentence, and a way back  
8   }  
9 }
```

► `location.search` is the address from the `?` on; `URLSearchParams` reads it as pairs; `get('id')` is `NULL` when there is no `?id=`. `find` gives `undefined` when no book matches, so `?? null` makes it the same word. **BOTH ARE AN EMPTY STATE**, not an error. Test: `book.html?id=b3` · `?id=nope` · no id. A link (cycle 2) writes the id with `encodeURIComponent(id)`, safe for any text.

THE ID IN THE ADDRESS: GET() AND ENCODEURIComponent()

THE ADDRESS	NEW URLSEARCHPARAMS(LOCATION.SEARCH).GET('ID')
<code>book.html?id=b4</code>	<code>'b4'</code>
<code>book.html?id=</code>	<code>''</code> — found, and empty
<code>book.html</code>	<code>null</code> — not found

► Writing the link: `encodeURIComponent('a b&c=d')` is `a%20b%26c%3Dd` — an id a user made can hold any character, and `&` or `=` would split it. `get()` decodes it back.

THERE ARE EXACTLY TWO WAYS TO KEEP A SCREEN UP TO DATE

PATCH IT

Each handler makes the change the user asked for, in the DOM, and keeps the data in step by hand.

Correct for any one action. Not correct for two.

REDRAW IT

Each handler changes the data. One function draws the whole screen from the data, every time.

Correct for any sequence, because there is no sequence.

Not a trick — giving up an optimisation you were never paid for.

THE SAME APPLICATION, TWICE

left: patched · right: redrawn · identical data, identical buttons

↺ reloadopen ↗

THE SAME APPLICATION, PATCHED ON THE LEFT AND REDRAWN ON THE RIGHT

add oneremove the first+1 on the firstsort by quantity20 random operations, both panelsredraw both from their datareset

PATCHED — EVERY HANDLER DRAWS

flour2

olive oil1

almond milk4

cheese1

30 lines across four handlers

agrees with its data

REDRAWN — EVERY HANDLER ONLY CHANGES DATA

flour2

olive oil1

almond milk4

cheese1

20 lines across four handlers

agrees with its data

The left panel is not written badly. Every handler in it is correct on its own, and it was tested one button at a time. What it cannot survive is the two hundred orderings of those buttons that nobody tested.

▶ Press each button once and both agree. Then **20 RANDOM OPERATIONS** — and then “redraw both”.

23

← TAUGHT · 22

READ THE TWO REMOVE HANDLERS, AND NOTHING ELSE

PATCHED

```
1 if (event.target.closest('button.drop')) {  
2   row.remove();  
3   document.querySelector('#shown').textContent = list.children.length;  
4   document.querySelector('#none').hidden = list.children.length > 0;  
5 }
```

Three things to remember, and it forgot the array.

REDRAWN

```
1 const row = event.target.closest('li[data-id]');  
2 const id = row.dataset.id;  
3 const books = getState().books;  
4 setState({ books: books.filter((book) => book.id !== id) });
```

One fact. There is nothing left to forget.

WHERE A ROW SITS IS NOT WHAT A ROW IS

POSITION

```
1 li.dataset.index = index;  
2 // ...  
3 const i = Number(row.dataset.index);  
4 const book = getState().books[i];  
5 book.copies += 1;
```

Correct until something moves — a removal, a search, a sort. Then it changes the wrong book and shows you the right one changing.

IDENTITY

```
1 li.dataset.id = book.id;  
2 // ...  
3 const id = row.dataset.id;  
4 books.find((book) => book.id === id);
```

Correct in every order, because the id does not move. The next slide changes the book it finds.

▶ A `data-` attribute is a note a row carries about itself; JavaScript reads it as `row.dataset....`. Carry **WHO** the row is, not **WHERE** it sits.

NOTHING IS WRONG UNTIL SOMETHING REORDERS THE LIST

the same six items · left keyed on position · right keyed on id

reloadopen

TWO LISTS, ONE KEYED ON WHERE A ROW SITS AND ONE ON WHAT IT IS

sort both by quantitysearch both for "il"clear the search+1 on the third row of eachremove the third row of eachredraw bothreset

KEYED ON POSITION — DATA-INDEX

flour	2
olive oil	1
almond milk	4
cheese	1
paprika	3
oat milk	2

nothing pressed yet

```
function bumpByIndex(row) {
  /* The row's position in the list ON SCREEN, used as a p
    COLLECTION. Those are the same number only while noth
  const item = A.items[Number(row.dataset.index)];
  if (!item) return null;
  item.qty += 1;
  row.querySelector('.qty').textContent = item.qty;
  return item;
}
```

KEYED ON IDENTITY — DATA-ID

flour	2
olive oil	1
almond milk	4
cheese	1
paprika	3
oat milk	2

nothing pressed yet

```
function bumpById(row) {
  const item = B.items.find((candidate) => candidate.id ==
  if (!item) return null;
  item.qty += 1;
  row.querySelector('.qty').textContent = item.qty;
  return item;
}
```

TIMES THE WRONG ITEM CHANGED

0

the two collections agree

Both lists start identical and stay identical while nothing reorders them. Sort first, then press a row button, and count.

▶ **PRESS +1 BEFORE YOU SORT.** The counter stays at zero. Now sort, and press it again.

A NEW ARRAY, EVERY TIME

```
1  setState({ books: [...books, book] });           // add
2  setState({ books: books.filter((b) => b.id !== id) }); // remove
3  setState({ books: books.map((b) => (b.id === id ? { ...b, copies: b.copies + 1 } : b)) }); // +1
```

► `push` works today. It mutates the array **ALREADY INSIDE STATE**, so `setState` is handed the object it already had — and everything that ever compares old to new stops being able to.



A COMPONENT: ONE BOOK IN, ONE ROW OUT

```
1 export function bookRow(book) {  
2   const li = document.createElement('li');  
3   li.dataset.id = book.id;           // who this row is  
4   // ... the title, genre, copies and two buttons, all from book  
5   return li;  
6 }  
7  
8 list.replaceChildren(...books.map(bookRow));
```

► A **COMPONENT** is a function that takes the data for one piece of the page and returns the element for it, touching nothing else: the same book in, the same row out. The list is the books mapped through it. `bookRow` has been in `render.js` since cycle 1; today it stops saying where it sits and starts saying who it is.

THE SAME COMPONENT AS A TEMPLATE LITERAL

```
1  const badge = (text) => `${esc(text)}</span>`;
2
3  const row = (book) => `- ${books.map(row).join('')}</ul>`;

```

► No side effects, no knowledge of where the result will be put, and `list` calls `row` calls `badge`. That is composition, with no framework in the room.

THE PRICE OF A TEMPLATE LITERAL IS SIX LINES

the same row four ways · identical markup · and one item with a hostile namereloadopen

A COMPONENT IS A FUNCTION: ITEM IN, MARKUP OUT

build the same row four ways

compose: list calls row calls badge

give one item a hostile name

reset

1 · STRING CONCATENATION

```
function concatRow(item) {
  return (
    '<li data-id="' +
    item.id +
    '"' +
    '>' +
    '<span class="name">' +
    item.name +
    '</span>' +
    '<span class="qty">' +
    item.qty +
    '</span>' +
    '</li>'
  );
}
```

flour2

2 · TEMPLATE LITERAL, UNESCAPED

```
function rawRow(item) {
  return `<li data-id="${item.id}">
    <span class="name">${item.name}</span>
    <span class="qty">${item.qty}</span>
  </li>`;
}
```

flour2

3 · TEMPLATE LITERAL, ESCAPED

```
function safeRow(item) {
  return `<li data-id="${esc(item.id)}">
    <span class="name">${esc(item.name)}</span>
    <span class="qty">${esc(item.qty)}</span>
  </li>`;
}
```

flour2

4 · CREELEMENT

```
function domRow(item) {
  const li = document.createElement('li');
  li.dataset.id = item.id;
  const name = document.createElement('span');
  name.className = 'name';
  name.textContent = item.name;
  const qty = document.createElement('span');
  qty.className = 'qty';
  qty.textContent = item.qty;
  li.append(name, qty);
  return li;
}
```

flour2

ARE THE FOUR OUTPUTS THE SAME?

all four produced identical markup

TIMES THE ITEM'S NAME EXECUTED

0

COMPOSITION

WAY

SOURCE LINES

THE ATTACKER'S CODE RAN

► **PRESS “HOSTILE NAME” FIRST.** Two of the four execute it. Which two?

SIX LINES, AND THEY ARE THE WHOLE PRICE

```
1 function esc(value) {  
2   return String(value)  
3     .replaceAll('&', '&amp;')  
4     .replaceAll('<', '&lt;')  
5     .replaceAll('>', '&gt;')  
6     .replaceAll('"', '&quot;');  
7 }
```

► & **FIRST**, or you escape your own escapes. And `String(value)` first, because a number has no `replaceAll`.

WHEN TO USE WHICH, AND IT IS NOT A MATTER OF TASTE

TEMPLATE LITERAL + `esc`

A component you want to READ.
Shape is visible; the whole row fits on the screen.

`createElement`

A row that needs an element reference, a `dataset`, or a listener attached to it directly.

EITHER ONE WITHOUT `esc`

An item name is a string a user typed. It reaches the HTML parser, and it is the shortest wrong answer in the week.

► This assignment uses `createElement`, because your rows carry a `data-id`.

ONE LISTENER, ON SOMETHING RENDER NEVER REPLACES

```
1 list.addEventListener('click', (event) => {  
2   const row = event.target.closest('li[data-id]');  
3   if (!row) return;  
4   // ... setState from row.dataset.id  
5 });
```

► This is **DELEGATION** (week 9): one listener on the parent catches the clicks of every row inside it. Render empties the list on every keystroke, so a listener on a row would die with it; this one lives on the list, which render never replaces.

A BOOK ADDED A MINUTE FROM NOW ANSWERS ITS +1 WITHOUT ANYONE WIRING IT.

CYCLE 2 · 26 MIN

THE HANDLER THAT DESCRIBES, BY ID

The fault is still there because the two row handlers were moved as they were: they paint. The obvious fix — delete from the array too — makes a **+1** land on the wrong book and paint a 5 where there was a 1. Then both handlers become one `setState` each, by `data-id`, in a new array.

1 I DO

13 MIN

5.5 on the slides before this one,
7.5 in the editor · live-code/02-
handler-that-describes ·
`render(getState())` from the console
brings a removed row back · splice
by `data-index` (wrong) · `data-id`,
filter, map · the title links to
`book.html?id=`

2 WE DO

4 MIN

you type steps 2–3 · compare with
checkpoint 2 · remove, +1 below it,
add — nothing moves

3 YOU DO

7 MIN

undo: a two-line spec, Claude
writes it or you do, you review —
the array you replaced, kept whole

4 DEBRIEF

2 MIN

"both" — `setState` and
`row.remove()` — passes every
screen test · ↗ depth

CHECKPOINT 2

YOUR SCREEN SHOULD LOOK LIKE THIS

ספריית ההשאלה

הרשימה

ספר

ספריית ההשאלה של הכיתה

שם הספר

עותקים

מדור

הוסף

מוציגים: 6

המפה והים

מסע

2

1+

הסר

אור בחלון

רומן

1

1+

הסר

שעון החול

מדע

3

1+

הסר

גשר על הירקון

רומן

1

1+

הסר

כוכבים בצהריים

מדע

4

1+

הסר

שביל החלב

מסע

2

1+

הסר

Compare — six rows, every title a link to `book.html?id=...`. Then the sequence: remove, `+1` on the row below, add — every number stays where it belongs. In `events.js`: no `row.remove`, no `textContent`, three `setState`.

YOU DO · AI AS PARTNER

UNDO IS THE ARRAY YOU REPLACED

REMOVE

```
1 const books = getState().books;
2 const gone = books.find((book) => book.id === id);
3 setState({
4   books: books.filter((book) => book.id !== id),
5   undo: { books, title: gone.title }, // the old array, whole
6 });
```

UNDO

```
1 const { undo } = getState();
2 setState({ books: undo.books, undo: null });
```

► Not a description of what was removed — **THE ARRAY BEFORE THE REMOVAL** (the title is only for the sentence “removed”). Undo is one assignment, and the book comes back in its place, because `filter` and `map` never touched the old array.

WHAT BELONGS IN THE OBJECT, AND WHAT DOES NOT

IN

```
1 const state = {  
2   books: [],           // not computable  
3   query: '',           // the user typed  
4   genre: 'all',        // the user chose  
5   order: 'title',      // the user picked  
6   undo: null,          // a past state  
7 };
```

OUT

```
1 const state = {  
2   visibleBooks: [],    // = filter + sort  
3   count: 0,           // = its length  
4   genres: [],         // = the genres of the books  
5   summary: {},        // = page three  
6 };
```

► One question per field: **CAN I COMPUTE IT FROM THE OTHERS?** The search text is a field, `state.query`; the list it filters is not. What can be computed is **DERIVED**: a function works it out each time it is needed, and it does not live here. That function is a **DERIVATION**.

HOW LONG A STORED ANSWER STAYS TRUE

four questions about one collection · stored on the left, derived on the right

↺ reloadopen ↗

FOUR QUESTIONS ABOUT ONE COLLECTION, STORED ON THE LEFT AND DERIVED ON THE RIGHT

add an item · updates everythingremove one · updates everythingquick fix: bulk import · updates the collection onlyrename a shelf · updates the collection onlyreset

STORED — FOUR FIELDS BESIDE THE COLLECTION

QUESTION	THE STORED ANSWER	ASKED AFRESH
count	3	3
total	7	7
shelves	cold, dry	cold, dry
heaviest	almond milk	almond milk

nothing changed yet

DERIVED — FOUR FUNCTIONS, NO FIELDS

count(items)3

total(items)7

shelves(items)cold, dry

heaviest(items)almond milk

Nothing here can go out of date, and the reason is not discipline. There is nothing here to go out of date.

STORED ANSWERS CURRENTLY LYING

0

the two panels agree

answering all four afresh, averaged over 1000 runs: 0.0006 ms. Drawing them once: 0.100 ms. Act 9 is where those numbers start to matter.

► The first two buttons keep every stored field correct.
THE THIRD ONE IS WRITTEN A MONTH LATER, BY SOMEBODY WHO ONLY NEEDED TO CHANGE THE COLLECTION.

SEARCH, FILTER AND SORT ARE NOT THREE BEHAVIOURS

```
1 export function selectVisible(state) {  
2   const query = state.query.trim();  
3   return state.books  
4     .filter((book) => book.title.includes(query))  
5     .filter((book) => state.genre === 'all' || book.genre === state.genre)  
6     .sort((a, b) =>  
7       state.order === 'copies' ? b.copies - a.copies : a.title.localeCompare(b.title, 'he'),  
8     );  
9 }
```

**They are one derivation, and
the list on screen is its output.**

SORTING A COPY, AND COMPARING LIKE A PERSON

A COPY, THREE WAYS

```
1 books.filter(match).sort(byTitle); // filter already made a new array
2 [...books].sort(byTitle);          // spread makes one
3 books.toSorted(byTitle);           // ES2023: never mutates
```

THE COMPARISON

```
1 ['b', 'a', 'B', 'A'].sort();           // A, B, a, b
2 ['b', 'a', 'B', 'A'].sort((x, y) => x.localeCompare(y, 'en')); // a, A, b, B
```

► The default compares UTF-16 code units, so every capital comes before every lower-case letter. `localeCompare` with a locale is the order a person expects — cycle 3 passes `'he'`. `toSorted` is Baseline, widely available since July 2023 (MDN).

THEN THE THREE HANDLERS ARE ONE LINE EACH

```
1 query.addEventListener('input', () => setState({ query: query.value }));  
2 genre.addEventListener('change', () => setState({ genre: genre.value }));  
3 order.addEventListener('change', () => setState({ order: order.value }));
```

► Here `query`, `genre` and `order` are the three controls on the page, found once with `querySelector`. Note what the third handler does **NOT** do: it does not sort anything. A sort handler longer than one line is sorting, and sorting is a derivation.



OPTIONS NOBODY WROTE, AND THE ONE EXCEPTION

```
1 export function selectGenres(state) {  
2   const genres = new Set(state.books.map((book) => book.genre)); // each genre once  
3   return [...genres].sort((a, b) => a.localeCompare(b, 'he'));  
4 }
```

```
1 function renderGenres(state) {  
2   const select = document.querySelector('#genre');  
3   const kept = select.value; // read before...  
4   select.replaceChildren(...options); // "all", then one option per genre  
5   select.value = kept; // ...put back after  
6   if (select.value === '') select.value = 'all';  
7 }
```

► Remove the last book of a genre and its option disappears — **NOBODY WROTE THAT LINE**. And the only place this week that reads the screen: a `<select>` whose children are replaced forgets its `select.value`, so it is read before and handed back after.

WHAT SELECT.VALUE DOES WHEN THE OPTION IS GONE

WHAT YOU DO

WHAT THE SELECT SAYS

replace its options while `x` is chosen

`value` is the first option; `selectedIndex` 0

assign a value no option has

`value` is `' '`; `selectedIndex` -1

then

“all” — the line cycle 3 types

```
if (select.value === ' ')  
  select.value = 'all'
```

- ▶ The honest limit: remove the last book of the chosen genre and the select falls back to “all” while `state.genre` still names the gone genre — the list is empty under a filter that says all. **THE FIX BELONGS IN STATE**, not in the select.

THE THIRD PAGE HAS NO DATA, AND THAT IS THE PROOF

STATE.JS · ONE MORE DERIVATION

```
1 export function selectSummary(state) {
2   const count = (genre) =>
3     state.books.filter((book) => book.genre === genre).length;
4   return {
5     total: state.books.length,
6     copies: state.books.reduce((sum, book) => sum + book.copies, 0),
7     byGenre: selectGenres(state).map((genre) => ({ genre, count: count(genre) })),
8   };
9 }
```

RENDER.JS · ONE ENTRY POINT

```
1 export function render(state) {
2   renderList(state);    // returns at once if #books is not here
3   renderBook(state);    // returns at once if #book is not here
4   renderSummary(state); // returns at once if #summary-count is not here
5 }
```

► `reduce` walks the array carrying a running total, here from `0`. The summary page holds no data and has no script of its own:

NO IF ASKS WHICH PAGE THIS IS. A fourth page is a region and a function.

CYCLE 3 · 25 MIN

SEARCH, FILTER AND ORDER ARE ONE DERIVATION

Three controls, wired to nothing. Not three handlers that each filter: three fields in state and one function, `selectVisible`. An order handler that stores its sorted result loses the next book added.

1 I DO13 MIN 5 on the slides before this one, 8 in the editor · live-code/03-one-derivation · selectVisible · selectGenres and the select.value exception (renderGenres: copy this) · a six-line order handler (wrong, copy this) · order in the derivation	2 WE DO4 MIN you type steps 1, 2 and 4 · compare with checkpoint 3 · a genre, then one letter: one row, and the select keeps its genre	3 YOU DO6 MIN two empty states, derived: nothing here yet, and nothing matched	4 DEBRIEF2 MIN a stored "visible" list works in every demo · ↗ depth
---	--	--	--

YOU DO · AI-FREE



CHECKPOINT 3

YOUR SCREEN SHOULD LOOK LIKE THIS

ספריית ההשאלה

הרשימה

ספר

ספריית ההשאלה של הכיתה

שם הספר

עותקים

מדור

הוסף

חיפוש

מדור

סדר

לפי שם

הכול

מוצגים: 6

אור בחלון

רומן

1

1+

הסר

גשר על הירקון

רומן

1

1+

הסר

המפה והים

מסע

2

1+

הסר

כוכבים בצהריים

מדע

4

1+

הסר

שביל החלב

מסע

2

1+

הסר

Compare — six rows by title, the genre filter with three genres. Then: pick the science genre and type one letter — one row, and the filter still says science; "most copies" and add a book with 3 — it lands in its place. Three handlers, one line each.

YOU DO · AI-FREE

THE EMPTY STATE IS A PROPERTY OF THE DATA, AND THERE ARE TWO OF THEM

```
1  const shown = selectVisible(state);  
2  none.hidden = shown.length > 0;  
3  none.textContent =  
4    state.books.length === 0  
5    ? 'no books yet – add the first one'  
6    : 'no book matches the search and the genre';
```

► “There is nothing here” sends the user to the add form. “Nothing matched” sends them to the search box.

ONE SENTENCE FOR BOTH IS A WRONG SIGN.



BREAK

10 MINUTES

Back at 95. Cycle 4 is the AI cycle — a chat window open, free or paid, and `ai-lab/04-add-a-filter/artifact/start/` in your editor. At 120, the surgery: no AI, no internet, alone.

CYCLE 4 · 25 MIN

ADD A FILTER — AND REVIEW WHAT CAME BACK AGAINST THREE LAWS

Prompt 1 is what almost everyone asks: one file pasted, and "I would rather not touch the others". What returns works when you tick the box — and stores a filtered list in state and hides rows in the handler. You catch it by doing one more thing with the box still ticked.

1 I DO 7 MIN 3 on this slide and the next, 4 in the chat · ai-lab/04-add-a-filter · P1 live · paste it · tick: 4 rows · the journal exemplar open	2 WE DO 6 MIN box ticked, then +1, search, add — find what breaks before it has a name	3 YOU DO 10 MIN your own P2 with the three laws written in · review before you run · one journal line	4 DEBRIEF 2 MIN a stored filtered list looks like a service · ↗ depth
--	---	--	--

YOU DO · AI AS PARTNER



PROMPT 1, AND WHAT CAME BACK

CLAUDE ADDED A FILTER. FIND THE TWO FAULTS.

THE PROMPT, WORD FOR WORD

```
I have a list of books on my page, and a checkbox  
with id="multi": "only books with more than one copy".  
Add a filter to it – when it is checked, only books  
with more than one copy are shown.  
Here is my events.js. I'd rather not touch the  
other files, they work.
```

```
<js/events.js – pasted whole>
```

► Tick it: four rows, “shown: 4”. Now keep it ticked and press `+1` on any book. **WHICH LAW BROKE FIRST?**



PROMPT 2 · THE LAWS IN THE PROMPT

WRITE THE RULE INTO THE PROMPT, AND THE ANSWER IS THREE LINES

WHAT P2 ADDS

1. One truth: nothing that can be computed is stored. The checkbox is one boolean field.
2. One painter: the handler touches no element and reads nothing from the screen.
3. One direction: the handler is one `setState`. The filter is one more `.filter` in `selectVisible`.

WHAT CAME BACK

```
1 // state.js
2 multi: false,
3 .filter((book) => !state.multi || book.copies > 1)
4
5 // events.js
6 multi.addEventListener('change', () =>
7   setState({ multi: multi.checked }));
```

► Ticked, then `+1` on a one-copy book: **IT APPEARS** — it has two now, and the derivation answers again on every draw. Nobody told the handler about the filter.

CHECKPOINT 4

THREE QUESTIONS, THREE YESES

You ran Prompt 1's answer

1 with the box still ticked

and did one more thing — `+1` , a search, an add.

Every one of the three laws is in your P2 as something you can check in the answer, and you checked it

2 by reading, before running

.

Your journal line says

3 what the browser showed

— a number of rows, a value in state. Shape: `prompt-journal-good.md` in your course materials.

SURGERY · 120–160

THE MINI-SURGERY, AND WHY THE FORMAT MATTERS MORE THAN THE MARKS

TASK	THE WORDING, AND IT IS WEEK 13'S	MINUTES
1 · repair	“The application does X. It should do Y. Find the cause and repair it.”	10
2 · extend	“Add Z. Change no behaviour that already works.”	10
3 · restructure	“Move this so that a future change to it is local. Behaviour identical.”	10

► **122–152 ALONE:** no AI, no internet, no neighbour; write the minutes on the sheet — finding versus fixing. **152–160 TOGETHER:** the walkthrough, with your sheet. Task 1 is cycle 2’s mistake, task 2 is cycle 3’s. Same three verbs in week 13, on **YOUR OWN** project, at three times the size; 30 percent of class assignment 10, **GRADED LIGHTLY ON PURPOSE.**

ASSIGNMENT · 160–175

CLASS ASSIGNMENT 10: THE SAME MOVE, ON THE PANTRY

1 Part one · the split · cycles 1 and 2

the sheet first, then three layers, four lines of `app.js`, `data-id`, handlers that only `setState`.

2 Part two · derived, not stored · cycle 3

`selectVisible`, `selectCategories`, one-line handlers, two empty states.

Parts three and four · cycles 1 and 2, you do

3 `item.html?id=`; undo as a snapshot. The summary is the same move on a third page: `selectSummary` and `reduce`, as on cycle 3's third-page slide.

FIFTEEN MINUTES IN CLASS

Run the four sequences in the starter and fill `observations-he.md` **before** you move a line. Commit it. The rest at home.

DIFFERENT NAMES, SAME MOVES

`#books` is `#list`, `selectGenres` is `selectCategories`. Nothing is copied; everything is recognised.

► **DUE BEFORE LESSON 11** — repo URL and SHA in Moodle.

FOUR WAYS THE SAME DISEASE SHOWS UP IN YOUR OWN CODE

- 1 Remove an item, add one — **the removed one returns.**
- 2 Search, then add something that does not match — **it appears anyway.**
- 3 Search, press `+1` on a row, clear the search — **a different row went up.**
- 4 Sort, then add — **the sort is gone.**

► All four are in the assignment starter's `js/app.js`. All four need **TWO ACTIONS IN THE RIGHT ORDER**, which is why nobody found them while writing it.

THREE QUESTIONS BEFORE YOU GO

- 1 Your state has `items` and `visibleItems`.

Which line of code will eventually be wrong, and who writes it?
- 2 You call `render(state)` twice in a row with the same object.

What must be true of the screen, and what does it prove?
- 3 A sort handler in your code is four lines long.

What is in the other three, and where do they belong?

“SO THIS IS REACT?”

WHAT REACT ADDS

A description of the screen you return instead of draw, and a diff so that redrawing everything only **writes** what changed.

WHAT REACT DOES NOT ADD

The model. **One state, one render, one direction** is the idea React was built to make cheap — it did not invent it.

WHY WE DO IT BY HAND

Because the part that is hard to learn is the model, and the part that is easy to learn is the library. Doing it once by hand makes the second one a Tuesday.

You are learning React thinking. The import statement is next semester's problem.

PERFORMANCE, HONESTLY

10 / 100 / 1000 / 5000 rows · measured on this machine, now

↺ reload

open ↗

WHAT A FULL REDRAW COSTS, AND WHAT ACTUALLY BREAKS FIRST

measure at 10 / 100 / 1000 / 5000 rows

list size100▼

redraw this list

redraw narrowed to 5 rows

patch one row

TIMINGS, ON THIS MACHINE, JUST NOW

ROWS	FULL REDRAW	REDRAW + LAYOUT	PATCH ONE ROW
not measured yet			

One frame at 60 Hz is 16.7 ms. Anything under it is invisible.

THE LIST

item 00

item 11

item 22

item 33

The text box is **INSIDE** the list and it redraws on every keystroke, which is exactly what a search box does. Type one character into it.

0

redraws that destroyed the focused element

0

redraws that threw away the scroll position. Scroll down first, then try each of the two redraw buttons — only one of them costs you the place you were.

WHERE THIS STOPS BEING FINE

a few hundred rows	fine
a control inside the redrawn region	not fine
a redraw on every keystroke over thousands of rows	not fine
an animation or a video inside the region	not fine

Only the first of those four is about speed. The fix for the other three is the same and it is not a framework: keep anything the user is holding on to **OUTSIDE** the part you replace.

► One frame at 60 Hz is 16.7 ms. **A FEW HUNDRED ROWS IS COMFORTABLY INSIDE IT**, and these numbers are this laptop's, not a blog post's.

WHERE A FULL REDRAW STOPS BEING FINE

SITUATION	FINE?	THE FIX, AND IT IS NOT A FRAMEWORK
a few hundred rows	yes	—
a control the user is holding, inside the region	no	move the control OUT of the region
a redraw per keystroke over thousands of rows	no	debounce, or render a window of the list
an animation or a video inside the region	no	move it out, or replace only what changed

Only the first row is about speed.